

🔒 Suad's Authentication Integration Guide

Overview

You'll be integrating the Password Reset Flow and Google OAuth into the Z-Events authentication system. The password reset backend is already implemented - you need to integrate it with the email service and build the frontend.

✔ What's Already Done

Backend - Password Reset Flow

- ✔ User model updated with resetPasswordToken and resetPasswordExpires fields
- ✔ POST /api/v1/auth/forgot-password endpoint created
- ✔ POST /api/v1/auth/reset-password endpoint created
- ✔ Token generation and validation logic implemented
- ✔ Swagger documentation added

Files Modified:

- User.ts
 - authController.ts
 - authRoutes.ts
-

🔧 Your Integration Tasks

Task 1: Connect Password Reset to Email Service (Coordinate with Justice)

Location:

backend/src/controllers/authController.ts line 147

Current Code:

```
// TODO: Send email with reset token (Justice will integrate this)
// await emailService.sendPasswordResetEmail(user.email, resetToken);
console.log(`Password reset token for ${email}: ${resetToken}`);
```

What to do:

1. Wait for Justice to complete the email service integration

2. Import the email service instance

3. Replace the TODO comment with actual email sending:

```
import emailService from '../services/emailService';
```

```
// In forgotPassword function:
```

```
await emailService.sendPasswordResetEmail(user.email, resetToken);
```

Task 2: Build Frontend Password Reset UI

2.1 Create "Forgot Password" Page

Location: frontend/src/pages/ForgotPassword/

Create:

- ForgotPassword.tsx
- ForgotPassword.css

Features:

- Email input field
- Submit button
- Success message display
- Error handling

API Call:

```
const response = await fetch('http://localhost:5000/api/v1/auth/forgot-password', {  
  method: 'POST',  
  headers: { 'Content-Type': 'application/json' },  
  body: JSON.stringify({ email })  
});
```

2.2 Create "Reset Password" Page

Location: frontend/src/pages/ResetPassword/

Create:

- ResetPassword.tsx

- ResetPassword.css

Features:

- Extract token from URL params (/reset-password/:token)
- New password input (with strength indicator)
- Confirm password input
- Submit button
- Success/error messages
- Redirect to login on success

API Call:

```
const response = await fetch('http://localhost:5000/api/v1/auth/reset-password', {  
  method: 'POST',  
  headers: { 'Content-Type': 'application/json' },  
  body: JSON.stringify({ token, newPassword })  
});
```

2.3 Add "Forgot Password?" Link to Login Page

Location: frontend/src/pages/Login/Login.tsx

Add a link below the login form:

```
<Link to="/forgot-password" className="forgot-password-link">  
  Forgot Password?  
</Link>
```

Task 3: Google OAuth Integration

3.1 Setup Google OAuth Credentials

1. Go to [Google Cloud Console](#)
2. Create a new project or select existing
3. Enable Google+ API
4. Create OAuth 2.0 credentials

5. Add authorized redirect URIs:

- <http://localhost:5173/auth/google/callback> (development)
- <https://your-production-domain.com/auth/google/callback> (production)

6. Save Client ID and Client Secret

3.2 Install Required Packages

Backend:

```
cd backend
```

```
npm install passport passport-google-oauth20 @types/passport @types/passport-google-oauth20
```

Frontend:

```
cd frontend
```

```
npm install @react-oauth/google
```

3.3 Backend Implementation

Create: backend/src/config/passport.ts

```
import passport from 'passport';
import { Strategy as GoogleStrategy } from 'passport-google-oauth20';
import User from '../models/User';
passport.use(new GoogleStrategy({
  clientID: process.env.GOOGLE_CLIENT_ID!,
  clientSecret: process.env.GOOGLE_CLIENT_SECRET!,
  callbackURL: '/api/v1/auth/google/callback'
}),
async (accessToken, refreshToken, profile, done) => {
  try {
    // Check if user exists
    let user = await User.findOne({ email: profile.emails?.[0].value });
```

```

    if (!user) {
      // Create new user
      user = await User.create({
        email: profile.emails?.[0].value,
        firstName: profile.name?.givenName || "",
        lastName: profile.name?.familyName || "",
        isVerified: true, // Google emails are pre-verified
        password: Math.random().toString(36), // Random password (won't be used)
      });
    }

    return done(null, user);
  } catch (error) {
    return done(error as Error);
  }
}

));

export default passport;

Add to:

backend/src/controllers/authController.ts

export const googleAuth = passport.authenticate('google', {
  scope: ['profile', 'email']
});

export const googleAuthCallback = async (req: Request, res: Response, next: NextFunction) => {
  try {
    const user = req.user as any;

```

```

// Generate JWT token
const token = jwt.sign({ userId: user._id }, authConfig.jwtSecret as string, {
  expiresIn: authConfig.jwtExpiresIn
} as any);

// Redirect to frontend with token
res.redirect(`${process.env.FRONTEND_URL}/auth/success?token=${token}`);
} catch (error) {
  next(error);
}
};

```

Add to:

backend/src/routes/authRoutes.ts

```

import passport from '../config/passport';
import { googleAuth, googleAuthCallback } from '../controllers/authController';
router.get('/google', googleAuth);
router.get('/google/callback',
  passport.authenticate('google', { session: false, failureRedirect: '/login' }),
  googleAuthCallback
);

```

Update:

backend/src/app.ts

```

import passport from '../config/passport';

```

```

// Add after other middleware

```

```

app.use(passport.initialize());

```

Update:

backend/.env

GOOGLE_CLIENT_ID=your_client_id_here

GOOGLE_CLIENT_SECRET=your_client_secret_here

3.4 Frontend Implementation

Update: frontend/src/pages/Login/Login.tsx

```
import { GoogleOAuthProvider, GoogleLogin } from '@react-oauth/google';
```

```
function Login() {
```

```
  const handleGoogleSuccess = async (credentialResponse: any) => {
```

```
    try {
```

```
      // Send credential to backend for verification
```

```
      const response = await fetch('http://localhost:5000/api/v1/auth/google', {
```

```
        method: 'POST',
```

```
        headers: { 'Content-Type': 'application/json' },
```

```
        body: JSON.stringify({ credential: credentialResponse.credential })
```

```
      });
```

```
      const data = await response.json();
```

```
      localStorage.setItem('token', data.token);
```

```
      // Redirect to dashboard
```

```
    } catch (error) {
```

```
      console.error('Google login failed:', error);
```

```
    }
```

```
  };
```

```
  return (
```

```
    <GoogleOAuthProvider clientId={import.meta.env.VITE_GOOGLE_CLIENT_ID}>
```

```
      {/* Existing login form */}
```

```
    <div className="google-login-container">
```

```
<GoogleLogin
  onSuccess={handleGoogleSuccess}
  onError={() => console.log('Login Failed')}
/>
</div>
</GoogleOAuthProvider>

);
}
```

Create:

frontend/.env

VITE_GOOGLE_CLIENT_ID=your_client_id_here

□ Testing Checklist

Password Reset Flow

- ☐ Test forgot password with valid email
- ☐ Test forgot password with non-existent email (should still return 200)
- ☐ Verify reset token is generated and saved to database
- ☐ Test reset password with valid token
- ☐ Test reset password with expired token (wait 1 hour or modify expiry)
- ☐ Test reset password with invalid token
- ☐ Verify password is actually changed after reset
- ☐ Test login with new password

Google OAuth

- ☐ Test Google login button appears on login page
- ☐ Test clicking Google login opens Google consent screen

- ☐ Test successful Google authentication creates new user
 - ☐ Test existing user can login with Google
 - ☐ Verify JWT token is generated after Google auth
 - ☐ Test redirect to dashboard after successful Google login
-

API Endpoints Reference

Forgot Password

POST /api/v1/auth/forgot-password

Content-Type: application/json

```
{  
  "email": "user@example.com"  
}
```

Response:

```
{  
  "message": "If an account with that email exists, a password reset link has been sent.",  
  "resetToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..." // Only in development  
}
```

Reset Password

POST /api/v1/auth/reset-password

Content-Type: application/json

```
{  
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",  
  "newPassword": "newSecurePassword123"  
}
```

Response:

```
{
```

```
"message": "Password reset successful. You can now log in with your new password."
}
```

∞ Coordination Points

With Justice (Email Service)

- Wait for email service to be ready before activating email sending in

`forgotPassword`

- Ensure

`sendPasswordResetEmail` method exists in `EmailService`

- Test email delivery in development environment

With Tony (Payments)

- No direct dependencies

With Elijah (SMS)

- Optional: Add SMS notification for password reset as backup to email
-

🔒 Security Notes

1. Never reveal if email exists - Always return same message for security
 2. Token expiry - Reset tokens expire in 1 hour
 3. One-time use - Tokens are cleared after successful password reset
 4. Password validation - Ensure new passwords meet minimum requirements (add validation)
 5. Rate limiting - Consider adding rate limiting to prevent abuse
-

📖 Resources

- [Google OAuth 2.0 Documentation](#)
- [Passport.js Google Strategy](#)
- [React OAuth Google](#)

✓ Definition of Done

- ☐ Password reset email integration complete
- ☐ Forgot password frontend page created
- ☐ Reset password frontend page created
- ☐ Google OAuth backend configured
- ☐ Google OAuth frontend integrated
- ☐ All tests passing
- ☐ Documentation updated
- ☐ Code reviewed and merged